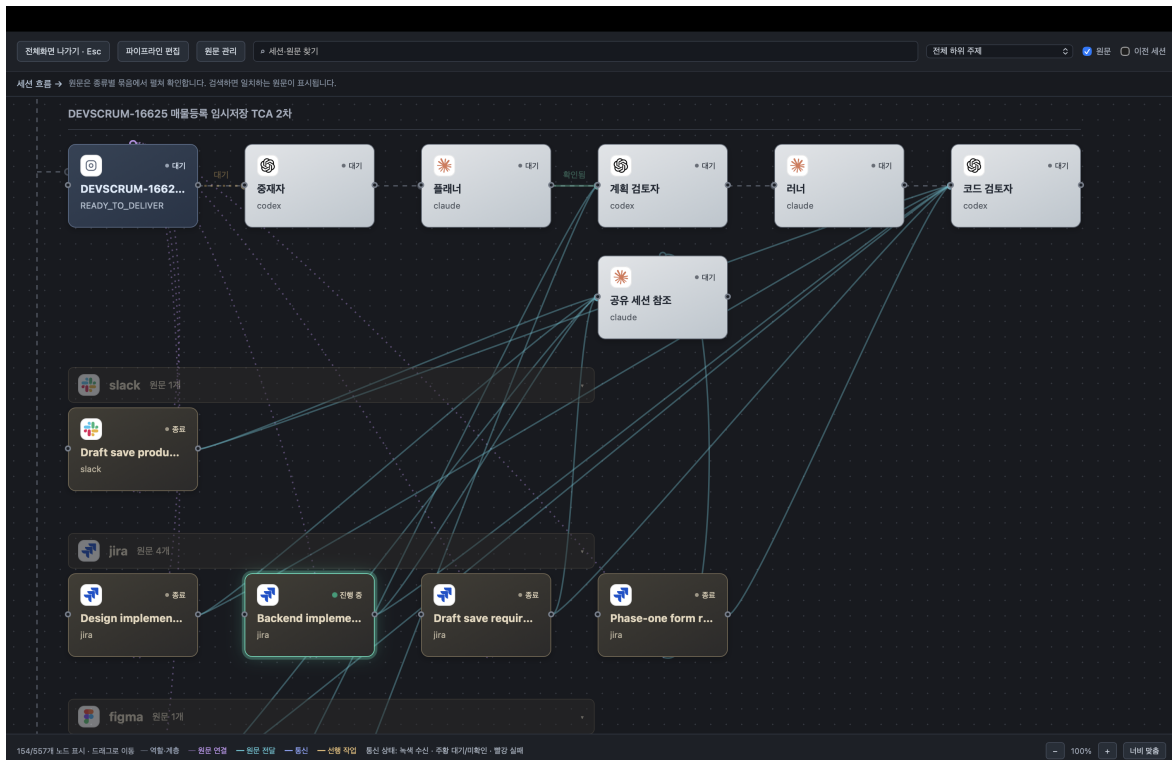


AI 개발 작업 관리

Claude와 Codex의 계획·구현·검토, 그리고 판단에 사용한 자료를 연결한 작업 공간.

Claude와 Codex를 함께 쓰면서 같은 요구사항을 다시 설명하고, 한 도구의 계획을 다른 도구에 복사하고, 수정된 코드를 다시 검토시키는 일이 반복됐다. 대화가 길어질수록 어떤 계획이 승인됐는지, 리뷰가 어느 코드에 대한 것인지 확인하기도 어려워졌다.



개발 배경과 설계 초점

이 도구에서 내가 설계한 핵심은 역할을 나누는 것보다 그 사이의 연결을 남기는 데 있다. 계획의 내용, 읽은 원문, 검토한 코드, 실제 커밋을 서로 연결해 다음 행동의 근거로 삼는다. AI가 구현을 수행하더라도 제품 결정과 전달 범위는 별도로 확인한다.

작업을 이어 가는 흐름

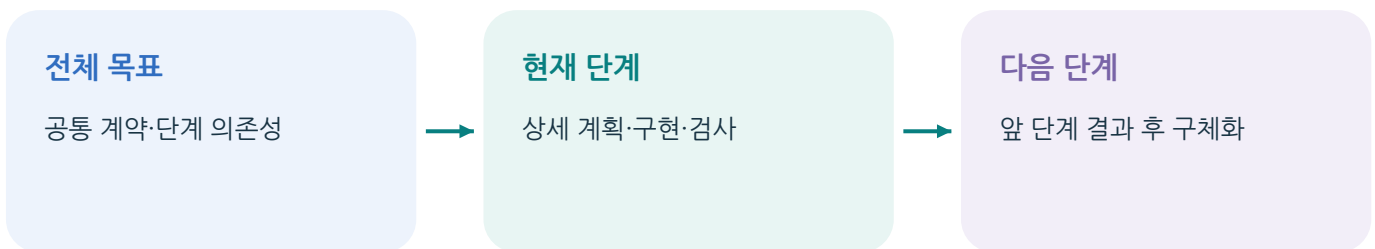
전체 목표는 먼저 정하되 지금 할 단계만 상세히 계획한다. 원문은 위키와 자료 목록에서 필요할 때 나눠 읽는다. 역할에 배정한 AI를 바꿀 수 있게 하고, 계획과 구현은 호환되는 세션에서 이어 간다. 수집·판단·실행·복구의 책임을 나누고 승인된 다음 행동은 서버가 이어 간다. 완료할 때는 검토한 코드와 실제 커밋을 대조한다.

세션 그래프: 역할별 참여자와 외부 원문 연결. 화면의 배경과 상태는 촬영 당시 기준.

전체 목표와 단계별 계획

전체 방향은 먼저 정하고, 실행할 단계는 앞 단계의 실제 결과를 바탕으로 구체화.

초기 흐름은 계획 단계에서 많은 요구사항과 코드를 한꺼번에 모았다. 입력이 커질 때 요약을 더 압축하거나 전달 상한을 높이는 것만으로는 해결되지 않았다. 어떤 근거가 빠졌는지 확인하기 어려웠고, 구현 시점에야 필요한 디자인 정보까지 계획에 들어갔다.



현재 필요한 원문은 나눠 읽고, 아직 판단할 수 없는 미래 단계는 개요로 보존.

전체 계획과 현재 단계의 분리

개편에서는 전체 계획과 현재 단계의 계획을 분리했다. 전체 계획은 목표·공동 계약·단계 사이의 의존 관계를 설명한다. 세부 구현은 앞 단계의 실제 결과를 확인한 뒤 구체화한다. 이미 완료된 작업의 판단을 다음 단계마다 다시 만들지 않는 것이 목적이다.

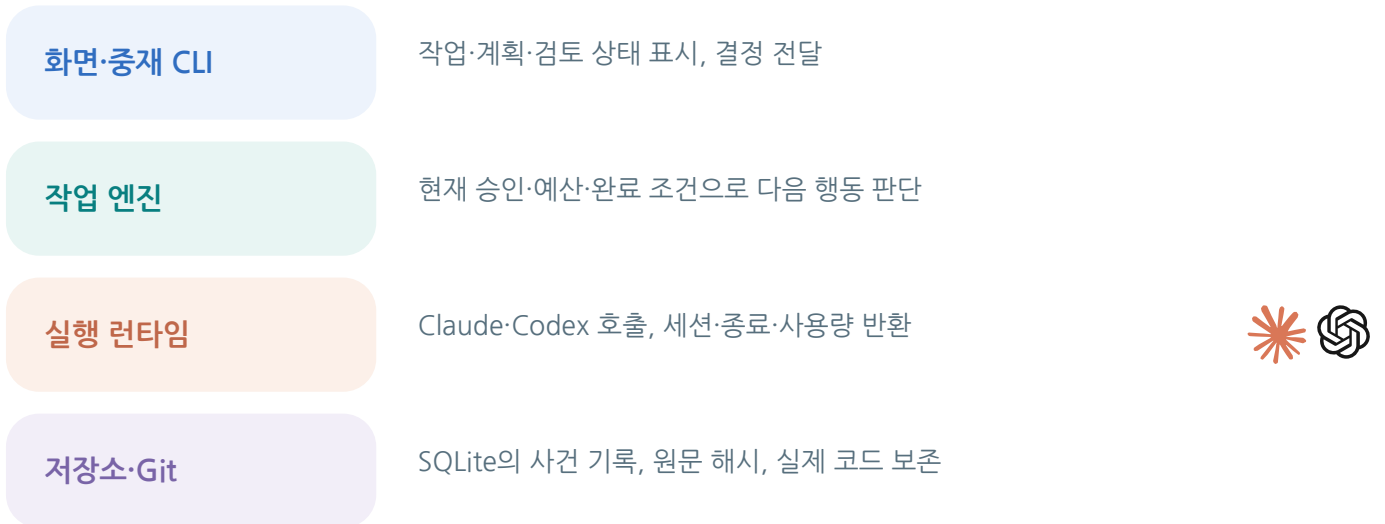
입력 크기와 작업 예산

한 번에 전달할 수 있는 크기와 전체 작업의 예산은 다른 문제다. 전자는 자료를 나눠 읽게 하는 장치이고, 후자는 비용과 실행 범위를 통제하는 장치다. 입력이 커졌다는 이유로 원문을 조용히 잘라 버리거나 새 세션으로 예산을 초기화하지 않도록 구분했다.

화면·엔진·실행 도구의 역할

상태와 권한을 판단하는 엔진, 모델을 호출하는 런타임, 사실을 보존하는 저장소.

React·Vite 대시보드는 작업·계획·리뷰·예산을 보여 준다. Node.js·TypeScript 서버는 상태 전이와 다음 행동을 결정한다. SQLite에는 승인, 역할 배정, 단계 결과, 실행 기록을 저장하고, 원문과 결과 파일은 내용 해시로 식별한다.



역할별 구현 책임

React·Vite는 화면을, Node.js·TypeScript는 서버를 구성한다. 공통 실행 런타임은 Claude·Codex CLI의 옵션과 출력 차이를 어댑터 뒤로 모은다. Git 작업 트리는 코드 변경을 격리하며, 원문과 결과 파일은 내용 해시로 식별한다.

공통 실행과 정책의 경계

엔진과 호스트 리뷰는 같은 실행 계약을 쓰지만 각자의 승인과 예산을 소유한다. 런타임이 운영 DB를 대신 수정하거나 리뷰 승인을 만들어 내지 않는다. 이렇게 나누면 공급자 실행 방식을 고쳐도 작업 승인 규칙까지 바꾸지 않아도 된다.

계층별 책임과 공통 계약

수집 사실과 작업 판단을 분리하고, 다음 행동은 엔진의 공통 조건으로 실행.

자료 수집, 작업 보류, 실행 복구, 다음 단계 시작이 서로의 내부 상태를 해석하면 같은 사실을 다르게 판단할 수 있다. 자료 하나를 읽지 못한 상태가 전체 작업 중지로 번지거나, 실행을 복구해도 다음 단계는 시작되지 않는 문제가 생겼다.

수집 어댑터

요청·응답·오류 등 실제로 관측한 사실

근거 관리

미수신·접근 실패·재수집 결과의 보존

구현자·리뷰어

확보한 근거로 할 일과 보류할 일의 판단

작업 엔진

같은 계획의 승인·복구·완료 조건 검사

진행 조정기

승인된 다음 행동과 실행 결과의 연결

수집부터 실행까지의 책임

수집 → 근거 관리 → 작업 판단 → 승인·완료 검사 → 다음 행동 실행으로 책임을 나눴다. 각 계층의 입력·출력을 타입과 공통 인터페이스로 정하고, 그 계약을 실제로 사용하는 경로에서 검사했다.

타입 검사와 복구 경로 검증

수집기는 제품 결정을 내리지 않고, 진행 조정기는 승인 조건을 복제하지 않는다. 타입 검사는 인터페이스의 일치를 확인하고, 실제 Git·공개 API·DB 재연결 검사는 중지와 복구 뒤에도 같은 계약이 지켜지는지 확인한다.

역할·프로필·배정·세션

역할은 책임, 프로필은 실행 설정, 배정은 연결 관계, 세션은 대화 기록.

설계자·구현자·검토자라는 역할은 허용할 행동과 결과 형식을 설명한다. 실행 프로필은 공급자·모델·추론 강도·공급자별 옵션을 담는다. 배정은 어느 작업에서 어떤 프로필을 쓰는지와 그 버전을 연결한다. 세션은 해당 배정의 대화 기록이다.



공급자 변경 전, 역할·옵션·이전 세션의 호환성을 확인.

배정 변경과 호환성

이 구분이 없으면 중재자를 Claude에서 Codex로 바꿀 때 역할과 무관한 설정까지 다시 만들어야 한다. 개편에서는 배정과 기능 지원 여부를 먼저 확인한 뒤 공통 실행 요청으로 변환한다. 전역 배정을 작업별 배정으로 덮어쓸 수 있지만, 기록 없이 설정을 바꾸지는 않는다.

새 공급자의 연결 조건

실제 어댑터는 Claude와 Codex 두 개다. 임의의 AI 도구를 연결하면 곧바로 동작하는 구조는 아니다. 새 공급자는 실행·권한·세션·사용량·결과 파싱 계약을 구현해야 한다. ultracode 같은 옵션도 공급자별 계약으로 남겨 공통 추론 강도와 혼동하지 않는다.

중재자의 확인 순서

대화의 마지막 문장보다 서버의 현재 상태·허용 행동·배정 신원을 먼저 확인.

중재자는 어떤 작업이 진행 중인지, 무엇 때문에 멈췄는지, 남은 질문과 예산이 무엇인지 확인한다. 이미 승인된 범위 안의 수정과 다음 준비된 단계를 조율하고, 서버에 저장된 자동 진행 상태를 확인한다. 제품 결정·인증·권한·예산처럼 실제로 부족한 조건은 이유를 남기고 요청한다.



이미 실행 중이면 중복 시작하지 않고, 실제로 필요한 결정만 사용자에게 요청.

중재자 교체와 상태 확인

대화의 마지막 문장만 보고 재개하지 않도록 현재 맥락 조회, 재개 상태 확인, 역할 배정 신원 확인 순서를 둔다. 중재자가 교체돼도 이전 승인과 질문이 사라지거나 이미 실행 중인 작업을 중복 시작하지 않게 하는 절차다.

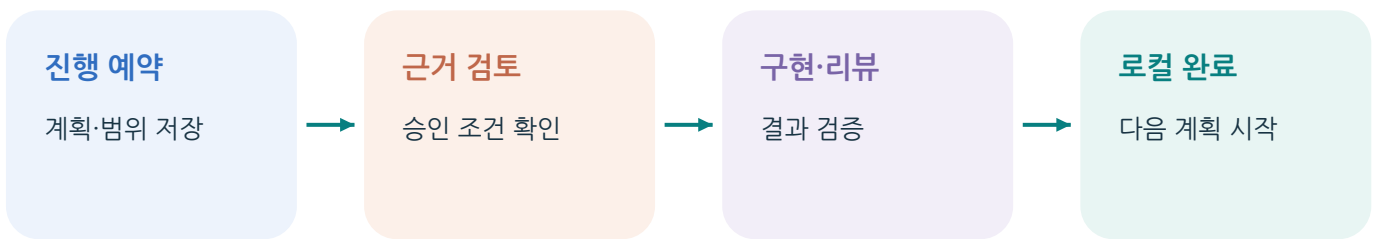
자동 진행의 권한 범위

자동 진행 정책은 매번 “계속할까요?”라고 묻는 비용을 줄인다. 그렇다고 구현자가 승인 범위를 넓히거나 리뷰 예산을 추가할 권한까지 갖는 것은 아니다. 로컬 완료, 공개 전달, 운영 설치 같은 행동은 해당 작업에 부여된 권한을 따라야 한다.

서버가 맡는 연속 실행

승인된 행동을 DB에 저장하고, 대화의 알림 수신 여부와 별도로 실행 결과를 추적.

단계가 끝났다는 알림만 보내면 중재 대화가 응답하지 않을 때 작업도 멈춘다. 서버의 진행 조정기는 현재 계획·범위·중재자·허용된 전달 범위를 저장하고, 작업 엔진의 공통 승인·완료 검사를 거쳐 다음 행동을 실행한다.



재시작 시 예약과 실제 실행 기록 대조

승인된 계획의 연속 실행

자동 진행을 승인한 계획은 근거 검토, 구현과 리뷰, 로컬 완료, 지정된 다음 단계의 계획 시작으로 이어진다. 새 단계의 계획 승인은 별도로 확인한다. 알림은 이 진행을 보여 주는 수단이며 실행 조건을 대신하지 않는다.

실행 상태와 재개 위치의 저장

실행 시작·정지·실패의 상태와 사건을 하나의 DB 트랜잭션으로 기록한다. 저장 중 종료돼도 확정된 기록을 기준으로 복구한다. 새 실패를 무한 재시도하거나, 종료된 실행이 새 계획의 상태를 덮어쓰지 않게 한다.

완료 결과를 기준으로 한 단계 분리

같은 결과를 만드는 설계·구현·검사는 한 단계 안의 체크리스트로 구성.

설계·구현·테스트를 각각 별도 단계로 만들거나 파일별로 단계를 나누면 같은 내용을 여러 번 설명하고 검토하게 된다. 개편에서는 한 결과를 완성하는 작업을 단계 안의 체크리스트로 묶는다. “이 단계를 끝내면 무엇을 확인할 수 있는가?”를 먼저 적는다.

파일 단위로 분리

타입 → 구현 → 테스트마다 인계

하나의 결과로 구성

파서의 입력·출력 계약 완성

검색 기능처럼 독립된 결과가 생길 때 다음 단계로 분리.

단계 분리의 판단 기준

분리는 앞 단계의 결과가 있어야 판단할 수 있는 경우, 독립적인 검증이 필요한 경우, 실패 시 따로 되돌릴 필요가 있는 경우로 설명한다. 단계 수나 코드 줄 수를 일률적으로 제한하지 않는다.

현재 단계의 완료 조건

현재 단계에는 완료 조건, 검사 방법, 예산, 해결해야 할 질문이 필요하다. 미래 단계는 개요로 남길 수 있다. 기본적으로 한 단계를 실행하며, 외부 정보나 사용자 결정을 기다릴 때만 의존하지 않는 준비된 단계로 진행할 수 있다. 예산 소진을 기다림으로 바꿔 다른 실행을 시작하지 않는다.

근거 변경과 판단 보존

변경된 근거의 영향만 다시 확인하고, 관련 없는 결과와 기존 구현 의무는 보존.

미래 단계를 추가·분할·병합하거나 순서를 바꿀 수 있다. 다만 이미 실행과 연결되거나 완료된 단계의 구조를 덮어 써 기존 결과의 의미를 바꾸지는 않는다. 진행 중인 단계의 전제가 바뀌면 변경 기록과 재계획을 연결한다.



재계획에 실패하면 대기 기록을 보존하고, 이전 계획의 늦은 응답을 차단.

변경 영향과 구현 의무의 보존

바뀐 근거가 현재 계획에 영향을 주는지 검토하고 관련 판단만 갱신한다. 기존 구현 의무는 ID뿐 아니라 내용·처분·근거까지 대조해 보존한다. 계획 이후에 수행할 구현과 검증을 현재 계획의 입력 부족으로 잘못 분류하지 않도록 구분했다.

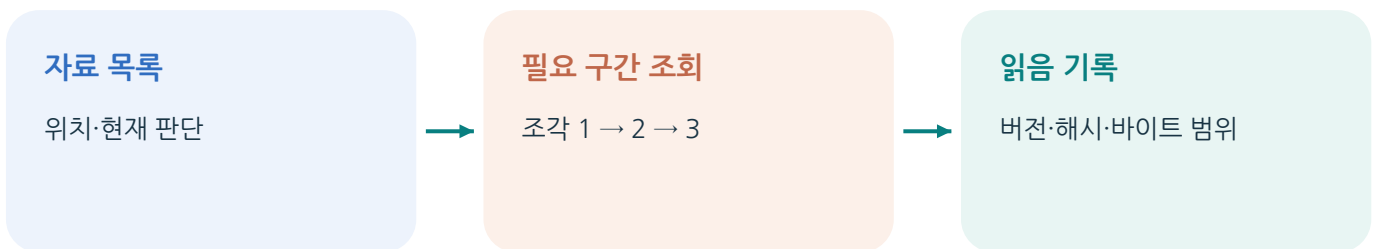
재계획 실패와 늦은 응답

재계획 중 오류가 나면 해당 변경을 대기 상태로 보존한다. 이전 계획으로 실행하던 응답이 늦게 도착해도 새 계획의 결과로 받아들이지 않는다. 같은 변경을 다시 처리할 때는 기존 기록과 예산을 이어 사용한다.

원문의 분할 조회와 읽음 기록

자료의 출처·버전·위치와 실제 전달 구간을 함께 보존.

계획 시작 시 목표, 진행 기록, 자료 목록과 참조를 전달한다. AI는 서버가 관리하는 읽기 도구로 파일·Git 변경·저장 자료를 조회한다. 전체 폴더와 대화 이력을 한꺼번에 넣는 대신 현재 질문을 해결하는 데 필요한 원문을 요청한다.



필수 읽기가 남은 응답은 중간 결과로 보존하며, 실제 전달 구간만 인용.

원문 조각의 출처와 전달 기록

읽은 조각에는 출처, 버전, 내용 해시, 바이트 범위, 다음 위치가 연결된다. 같은 문서의 앞부분을 반복해서 보내거나 버전이 다른 조각을 하나의 원문처럼 합치지 않도록 확인한다. 실제로 전달된 조각만 근거로 인용할 수 있다.

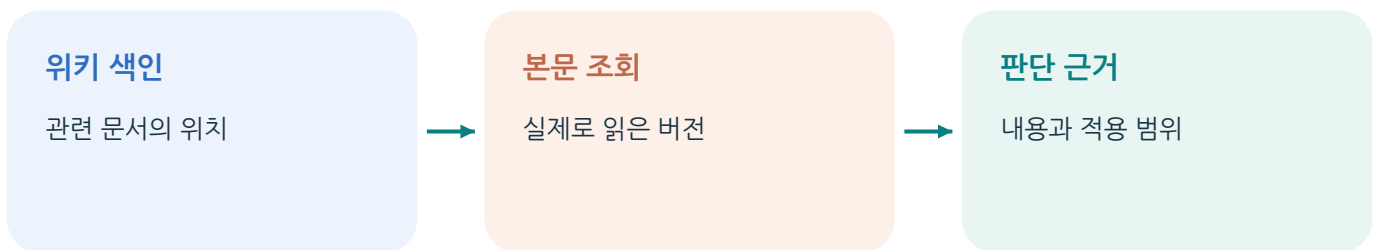
컨텍스트 손실의 복구 조건

누적 입력 크기만으로 강제 요약이나 새 세션을 만들지 않는다. 실제 컨텍스트 한도나 세션 손실이 확인됐을 때만 제한된 체크포인트 복구를 사용한다. 공급자·모델·설정과 누적 예산을 유지한다. 자료를 읽은 기록이 사라졌다면 재조회 이유를 남긴다.

위키 검색과 실제 근거의 구분

색인에서 찾은 문서와 실제로 읽고 판단에 사용한 원문을 구분.

위키 색인에는 문서의 위치와 관련 주제가 있다. 처음에는 관련 문서 일부만 선택하고, 더 필요한 자료를 질의해 읽는다. 모든 위키를 프롬프트에 붙이거나 색인에 나타난 모든 문서를 판단 근거로 등록하지 않는다.



검색 정확도 · 근거 최신성 · 전달량 · 실제 비용은 각각 별도로 확인.

실제 읽은 문서의 기록

실제로 조회한 문서의 경로·버전·내용을 기록한다. 읽지 않은 문서가 바뀌었다는 이유로 현재 작업을 모두 무효화하지 않는다. 반대로 읽은 원문이 바뀌면 관련 판단을 다시 확인해야 한다.

새 원문의 해석과 검색 비용

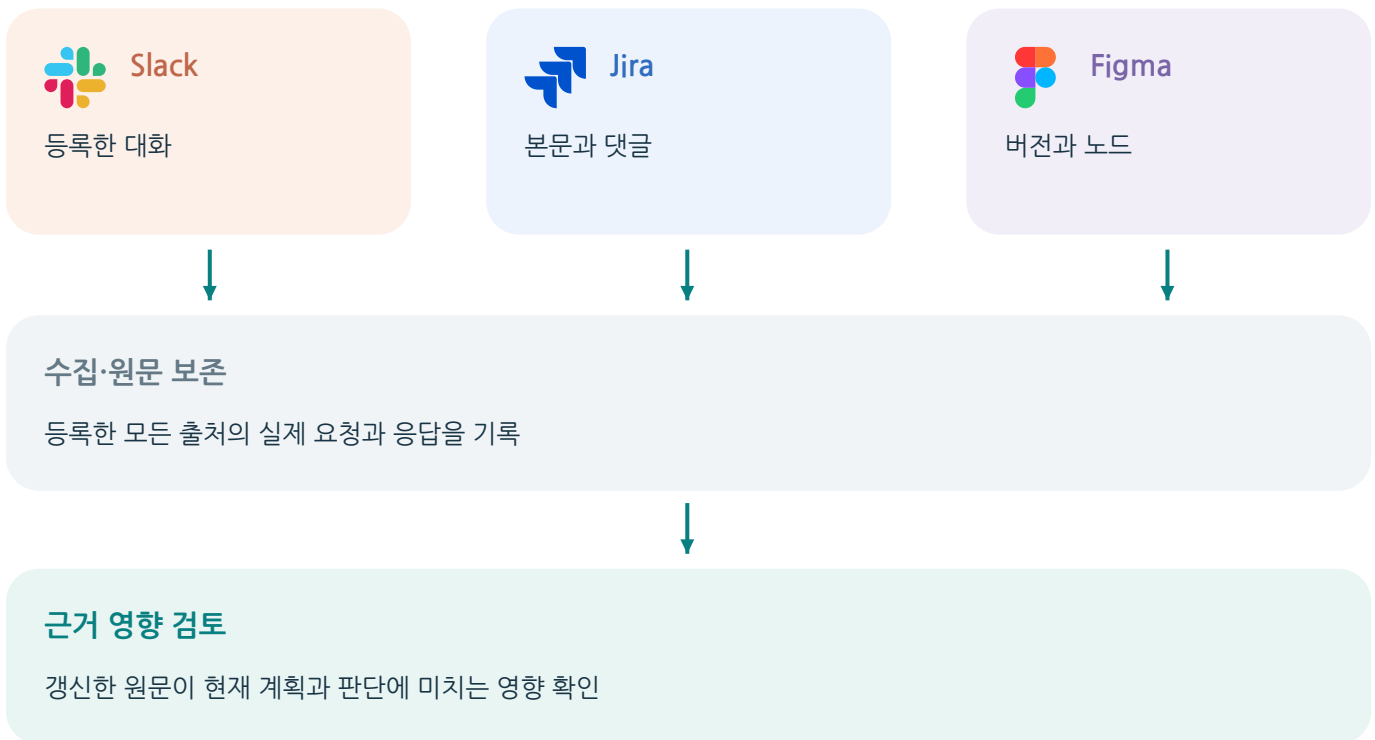
위키 본문의 해시와 외부 원문의 해시는 다른 정보를 가리킨다. 외부 자료가 갱신되면 “확인이 필요하다”는 상태를 남긴다. 새 내용이 질문인지 결정인지, 어느 플랫폼에 적용되는지, 기존 근거와 충돌하는지 판단한 뒤 결론을 고쳐야 한다.

검색 정확도가 좋아졌다고 토큰이 줄었다고 말할 수는 없다. 더 관련 있는 문서를 많이 선택하면 전달량이 늘 수도 있다. 위키 수정 제한도 경로·상태·내용 해시와 역할 권한을 확인한 뒤 반영하며, 폴더 전체를 임의로 고칠 권한을 주지 않는다.

Slack·Jira·Figma 자료 흐름

등록한 출처의 변경 감지와 모델에 전달할 근거의 판단을 분리.

Slack·Jira·Figma 링크를 등록하면 REST 수집기가 기본 15분 간격으로 조회 시점을 확인한다. 시작할 때 밀린 조회도 확인한다. 닫힌 작업에서만 사용하는 출처는 정기 조회 대상에서 빠진다. 외부 자료가 없는 작업에는 조회할 대상이 없다.



수집과 모델 전달의 분리

수집 자체는 LLM을 호출하지 않는다. 변경분은 실행 전 근거 영향 검토와 시작·재개 입력에 연결한다. 커넥터 방식은 해당 호스트의 자료 수집이 필요하며, REST 인증과 커넥터 인증을 자동으로 같은 것으로 취급하지 않는다.

Figma 확인 시점과 수집 실패

계획 단계는 화면별 링크를 디자인의 위치로 보관한다. 레이아웃·크기·간격·폰트·색상·노드 트리·스크린샷을 모아 계획에 옮기지 않는다. 구현자가 해당 화면을 만들 때 필요한 자료를 직접 확인한다. 링크나 캐시된 이미지가 있다는 사실만으로 디자인을 검증했다고 판단하지 않는다.

자료 변경이나 장기간 미확인은 실행·결과 수락 전 재검토가 필요하다. 접근 거부와 요청 제한을 최신 상태의 근거로 쓰지 않는다. 같은 본문을 다시 전달하지 않아도 API 조회나 공급자의 캐시 입력 비용은 남을 수 있다.

자료 누락과 후속 작업

부족한 자료에 의존하는 부분만 후속 목록으로 넘기고, 확보한 근거의 범위에서 진행.

외부 자료에 접근할 수 없다고 확인된 요구사항까지 모두 지우면 작업이 반복해서 멈춘다. 읽지 못한 자료와 그 자료가 필요한 동작을 후속 목록에 남기고, 확보한 근거로 진행할 수 있는 범위를 구현자와 리뷰어가 함께 확인한다.

확보한 근거

확인된 범위의 구현과 리뷰 진행

부족한 자료

의존 동작·오류를 후속 목록에 보존

공백 해소: 같은 요청의 성공 응답 확인. 필수 검사 실패와 제품 결정 부족은 별도 유지.

미수신과 접근 실패의 기록

요청했지만 응답이 없었던 경우와 실제 접근 오류를 구분한다. 둘 다 수집 성공이 아니다. 새 요청을 보내면 수집 상태를 다시 추적하고, 같은 요청의 성공 응답을 확인한 뒤에만 공백을 해소한다.

호스트 기록과 모델 판단의 경계

호스트는 누락을 저장하지만 리뷰어가 하지 않은 판단을 만들어 넣지 않는다. 구현자와 리뷰어가 같은 보류 목록을 읽고, 후속 단계에도 미해결 항목이 전달되게 했다. 링크를 지웠다는 이유만으로 누락이 해결되지는 않는다.

계획·구현·리뷰의 세션 연결

호환되는 대화와 유효한 읽음 기록을 이어 사용하고, 검토자의 판단 맥락은 별도로 유지.

설계자가 이미 코드와 요구사항을 이해했는데 구현을 새 세션에서 시작하면 판단 근거를 다시 요약하고 전달해야 한다. 개편에서는 계획과 구현의 배경이 호환될 때 같은 대화를 이어 사용한다. 원문 전체를 다시 붙이는 별도 전달 문서를 필수로 만들지 않는다.



설계·구현 대화

계획 → 구현
현재 배경과 호환될 때 재개



독립된 검토 대화

첫 리뷰 → 수정 후 리뷰
미해결 지적과 변경분 확인

재사용 전 작업·역할·세션·원문 버전·전달 구간의 일치 확인.

대화를 이어 쓰는 조건

이 연결은 세션 이름만으로 결정하지 않는다. 공급자, 참여자, 역할 배경과 실행 조건을 확인한다. 호환되지 않는 세션을 억지로 재개하거나 임의의 다른 모델로 대체하지 않는다.

검토자의 독립성과 재사용 범위

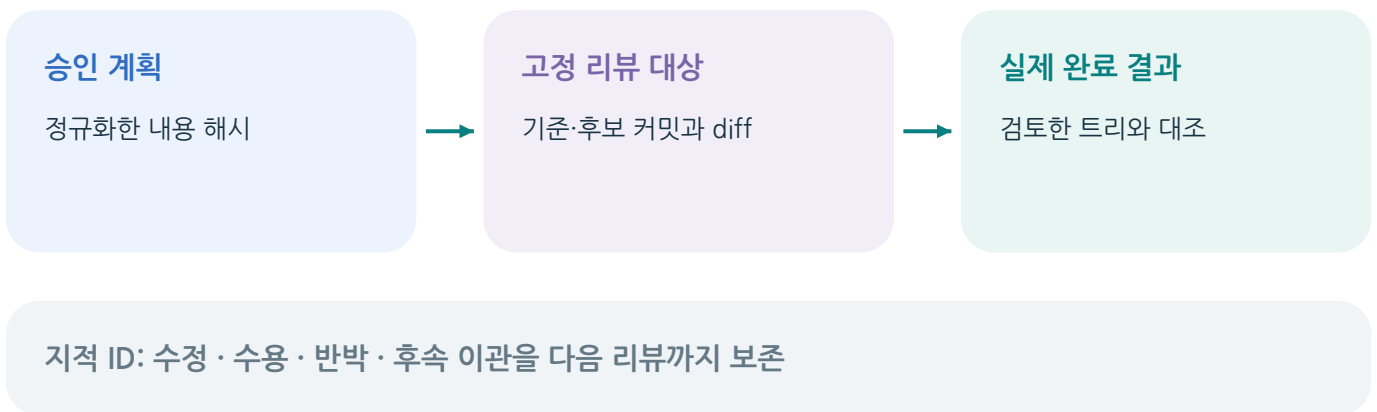
검토자는 고정된 계획과 구현 결과를 독립적으로 본다. 첫 리뷰는 전체 변경을 확인하고, 수정 후에는 같은 리뷰 맥락에서 바뀐 부분과 미해결 지적을 확인한다. 범위를 넓혀야 한다면 새 반증이나 공통 계약의 영향처럼 이유가 있어야 한다.

실패·취소된 전달은 읽음으로 기록하지 않는다. 사용량 확인 때문에 보류한 응답은 같은 입력·세션·원문 버전과 전달 구간을 대조해 재사용한다. 새 결정이나 원문 변경이 있으면 다시 확인한다. 실제 토큰 절감률은 같은 조건의 비교 실행으로 측정해야 한다.

승인 계획과 검토 코드의 동일성

계획 내용, 리뷰 대상, 실제 결과를 각각 고정하고 같은 대상인지 대조.

설계자와 검토자가 서로 다른 계획을 보고 동의하는 일을 막기 위해 정규화한 계획의 SHA-256을 사용한다. 현재 승인도 그 계획에 연결한다. 해시가 같다는 사실은 같은 내용을 가리킨다는 뜻이며, 계획이 옳다는 증명은 아니다. 타당성은 별도 검토가 맡는다.



계획 동일성과 리뷰 지적의 추적

리뷰에는 지적 ID와 처리 상태를 남긴다. 수정·수용·반박·범위 밖 이관을 구분하고, 응답에서 지적이 빠졌다고 해결된 것으로 처리하지 않는다. 불완전한 리뷰는 이전 지적을 지우거나 변경분 기준을 전진시키지 못한다.

고정된 코드와 후속 리뷰

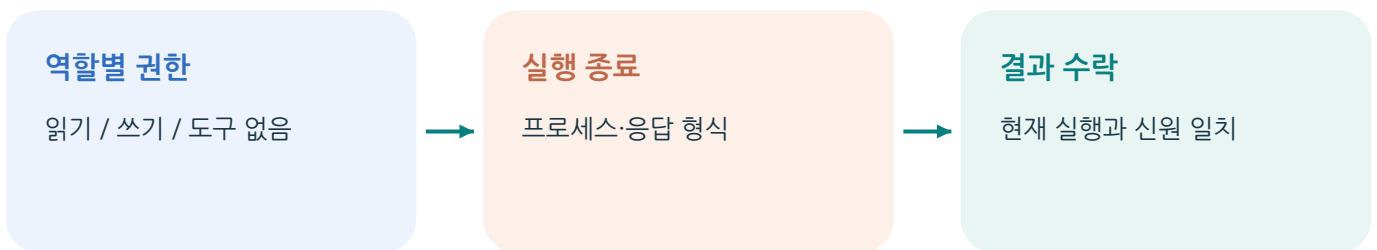
기준 커밋, 후보 커밋, 파일 목록과 변경 내용을 고정한 뒤 리뷰한다. 수정 중인 작업 폴더를 동시에 검토하면 리뷰가 끝났을 때 실제 코드가 달라질 수 있다. 후속 리뷰는 바뀐 지적과 영향을 받는 호출부를 확인하며, 매번 전체 작업을 처음부터 재감사하지 않는다.

범위 밖 개선은 다음 작업에 넘길 근거로 보존한다. 작은 개선을 계속 현재 작업에 포함해 리뷰가 끝나지 않는 흐름을 피하기 위한 선택이다.

역할별 권한과 실행 종료

실행 권한·프로세스 종료·결과 형식을 모두 확인한 뒤 결과 수락.

계획·리뷰는 읽기, 구현은 쓰기, 단순 확인 응답은 도구 권한 없이 실행하는 계약을 사용한다. 파일 수정 가능 여부를 특정 공급자 이름에 묶지 않는다. 해당 역할과 실행 도구가 요청한 권한을 지원하는지 먼저 검사한다.



취소 후 도착한 응답, 잘린 JSON, 미제공 사용량을 정상 완료로 합치지 않음.

실행 종료와 결과 형식의 확인

모델이 “완료”라고 출력했더라도 프로세스가 정상적으로 끝났는지와 결과 형식이 유효한지를 확인해야 한다. 출력 일부만 읽고 성공을 확정하면 뒤늦게 나온 오류나 잘린 응답을 놓칠 수 있다.

취소한 실행의 응답 차단

취소·재계획·재시작이 일어날 때 실행 신원과 상태를 대조한다. 이전 실행이 늦게 끝나도 최신 계획의 결과나 승인 상태를 덮어쓸 수 없다. 프로세스를 정리할 때도 PID만 보지 않고 시작 신원을 확인해 다른 프로세스를 잘못 종료하지 않도록 한다.

진행 표시와 실제 작업의 구분

실행 시간·문장 증가·완료 선언을 서로 다른 증거로 확인.

계획·리뷰·구현 결과를 파일로 남기면 대화가 길어져도 필요한 원문을 다시 읽을 수 있다. 다만 별칭 파일이 다른 내용으로 교체될 수 있으므로 내용 해시로 고정된 자료를 기준으로 삼는다. DB의 참조와 실제 파일을 읽을 때 해시를 대조한다.

실행 중 표시

현재 실행의 결과와 저장된 사건 기록 확인

문장 증가

읽은 자료·실제 파일·해당 턴의 결과 확인

완료 응답

검사·리뷰·로컬 커밋의 연결 확인

저장된 상태와 재연결

화면은 서버의 실행 기록과 이벤트를 받아 진행 상태를 보여 준다. 실행 시간이나 누적 문장이 늘어난 것만으로 진척을 판단하지 않는다. 현재 턴의 결과·실제 파일·읽은 자료를 대조하며, 재연결 뒤에도 저장된 상태를 확인한다.

검증 근거별 확인 범위

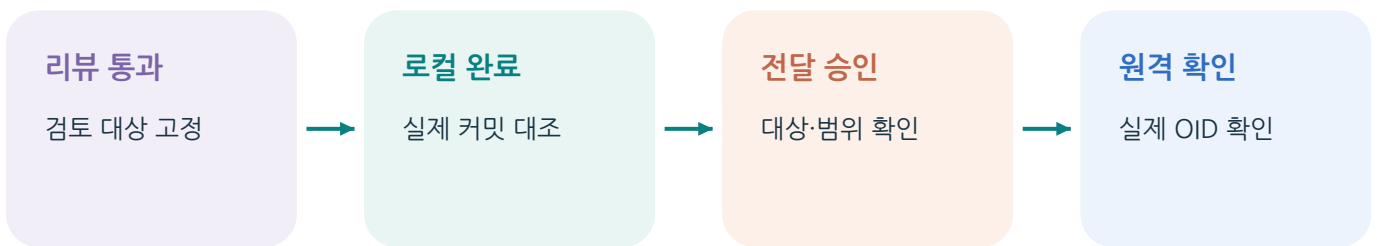
승인한 경로와 변경 규칙을 벗어났는지 확인할 수 있다. Swift 주석·문자열을 구분해야 하는 검사에서는 단순 문자열 치환으로 판단하면 오탐과 누락이 생긴다. 범위 규칙을 적용할 때도 해당 언어의 표현을 고려해야 한다.

서로 다른 증거를 하나의 성공 표시로 합치지 않는 것이 이 설계의 중요한 제한이다.

로컬 완료와 원격 전달

검토한 결과를 로컬 커밋에 연결하고, 원격 반영은 별도 OID로 확인.

단계가 끝나려면 승인된 계획과 리뷰 결과가 실제 로컬 커밋에 연결돼야 한다. 다음 단계는 그 커밋과 결과 묶음을 받는다. 모델이 보고한 커밋 문자열만 믿지 않고 기준 커밋과의 관계, 트리, 작업 상태를 확인한다.



푸시 응답이 유실되면 재실행에 앞서 원격 커밋과 기존 실행 기록을 대조.

완료 커밋과 단계 통합

갈라진 결과는 충돌 해결과 리뷰 뒤 부모 관계를 보존해 통합한다. 조건부 계획에서 변경이 필요 없었다면, 검토한 기준 커밋을 결과로 사용한다. 허용 경로는 수정 의무가 아니며, 변경이 없는 경우도 현재 승인·검토한 트리·선행 결과를 확인한다.

푸시 응답 유실의 복구

네트워크 오류로 결과를 받지 못해도 원격에는 이미 반영됐을 수 있다. 먼저 원격 커밋을 확인하고 저장된 실행 기록과 대조한다. 이미 반영된 작업을 중복 실행하지 않고 전달 상태를 복구한다.

파일럿에서는 격리된 bare 원격으로 이 흐름을 확인했다. 이 결과가 임의의 실제 서비스 배포와 운영 데이터 변경까지 검증했다는 뜻은 아니다.

사용량 기록과 실행 한도

원문 읽기 횟수, 논리적 리뷰, 실제 모델 실행을 서로 다른 값으로 기록.

리뷰가 원문을 여러 번 나눠 읽는다고 매번 별개의 리뷰를 시작한 것은 아니다. 읽기와 최종 판단을 하나의 논리적 리뷰로 연결할 수 있어야 큰 자료도 정상적으로 검토할 수 있다. 동시에 실제 모델 호출마다 발생한 사용량과 시간은 빠짐없이 기록해야 한다.

한 번의 논리적 리뷰

여러 원문 읽기 → 판단·보완 → 결과

실제 사용량 기록

호출별 입력·캐시·출력·실행시간

예산·공급자 제한은 별도 적용. 누락 사용량은 0으로 만들지 않고 미제공으로 보존.

논리적 리뷰와 실제 실행 횟수

수정과 확인 응답이 추가되는 경우에는 해당 정책에 따라 리뷰 예산을 사용한다. 실패한 실행을 성공처럼 지우거나, 새 커밋·새 세션·서버 재시작으로 같은 작업의 사용량을 초기화하지 않는다.

누적 사용량과 적용 한도

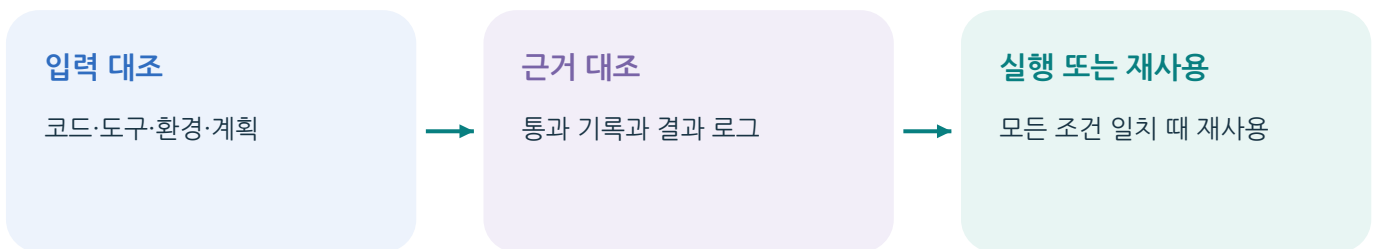
기본 정책은 시간·토큰 사용량을 기록한다. 사용자가 정한 예산·횟수 한도와 공급자의 사용 제한은 따로 적용한다. 중단 이유와 재시도 시점을 보존하며, 취소되거나 오래된 실행의 재시도가 새 계획을 덮어쓰지 못하게 한다.

9월 파일럿에서는 당시 설정에 따라 예산과 리뷰 횟수를 추가했다. 사용량이 누락되면 0으로 채우지 않는다. 현재는 해당 호출과 저장 응답을 보존하며, 미확인 사용량을 인정한 재개 권한도 그 호출에 연결한다.

검사 결과의 재사용 조건

확인된 정적 검사 한 종류의 입력·도구·환경·계획·로그를 함께 대조.

현재 재사용 경로는 알려진 Swift 동시성 정책 정적 검사 한 종류를 대상으로 한다. 관련 입력, 검사 도구, 실행 환경, 승인 계획과 결과 로그를 해시로 확인한다. Git 추적 대상이 아닌 관련 Swift 파일도 입력 집합에 포함해야 한다.



적용 사례: Swift 동시성 정책 정적 검사. 단위 테스트·빌드·UI 검사는 별도 확인.

검사 결과와 입력의 일치

검사 결과를 저장하는 도중 중단됐다면 다음 실행에서 완료 여부와 근거를 복구한다. 비교할 근거가 없다는 상태를 “변경 없음”으로 처리하지 않는다. CLI가 제출한 실행 보고서와 독립적으로 확인한 실행 증거의 차이도 남는다.

앱 실행 경험에서 정한 검증 범위

Swift 6 전환 과정에서는 빌드 로그 일부나 재사용된 산출물만 보고 성공을 판단할 위험이 드러났다. WebKit 종료 과정의 오류처럼 컴파일 뒤 실행에서 나타나는 문제도 있었다. 운영체제 로그와 앱 프로세스 오류를 구분하지 않으면 잘못된 실패를 추적할 수 있었다.

따라서 정적 검사 캐시를 전체 단위 테스트·빌드·UI 검증의 대체물로 확장하지 않았다. 효율보다 검증 결과가 의미하는 범위를 먼저 고정한 선택이다.

9월 파일럿의 실행 경로

첫 실패는 보존하고, 엔진 중간 수정 없이 새 파일럿에서 다섯 단계와 원격 전달까지 확인.

첫 파일럿은 예산 처리와 계획 확정 경로의 결함으로 중단됐다. 중단 기록을 유지한 채 관련 계약을 수정하고 검토했다. 그 실행을 이어 붙여 성공으로 만들지 않고 새 파일럿에서 전체 흐름을 다시 확인했다.



다섯 단계의 실행 결과

두 번째 파일럿은 실제 Claude·Codex CLI로 수행했다. 진행 중 엔진 파일을 바꾸지 않았고, 비교 대상 243개 파일의 해시가 유지됐다. 마지막에는 모든 단계를 닫고 최종 통합 결과를 격리된 bare 원격에 전달했다.

대상 검사와 실패 검출

최종 대상 프로젝트의 검사 77개가 통과했고, 고의로 필요한 export를 제거한 반대 검사에서는 실패했다. 통과 숫자만으로 판단하지 않고 검사가 실제 결함을 거부하는지도 확인했다.

파일럿 사용량과 검증 범위

9월 파일럿의 실행량과 10월 엔진 후보 검사 결과를 나눠 기록한 검증 사례.

9월 파일럿은 실제 Claude·Codex CLI를 사용해 다섯 단계를 수행하고 격리된 원격에 결과를 전달했다. 아래 수치는 그 실행에서 수집한 관측값이다. 비교 조건이 같은 이전 실행은 없으므로 이 값을 비용이나 시간의 절감률로 해석하지 않는다.

48회

실제 모델 실행

5개 단계

완료와 원격 전달

6,566초

호출별 실행시간 합계

입력 22,602,633 토큰 / 그중 캐시 입력 18,134,381 토큰

출력 620,113 토큰 · 캐시 입력은 전체 입력에 포함된 값

캐시 입력과 실행시간의 해석

캐시 입력은 전체 입력에 포함된 값이며 다시 더하지 않는다. 실행시간은 호출별 시간의 합계로, 작업 시작부터 완료까지의 경과 시간과 다르다. 실행 횟수·전달량·비용을 하나의 성능 지표로 합치지 않고 각각의 의미와 함께 보존한다.

공개 코드 검사와 파일럿의 차이

10월 5일 기록한 엔진 후보 검사에서는 개발본 2,417개와 호스트 리뷰 Python 221개가 통과했다. 예상 실패 2개·제외 7개를 별도로 집계했고, TypeScript 검사·웹 빌드·공식 리뷰도 통과했다. 이 결과는 아래 파일럿과 서로 다른 검증 범위를 가진다.

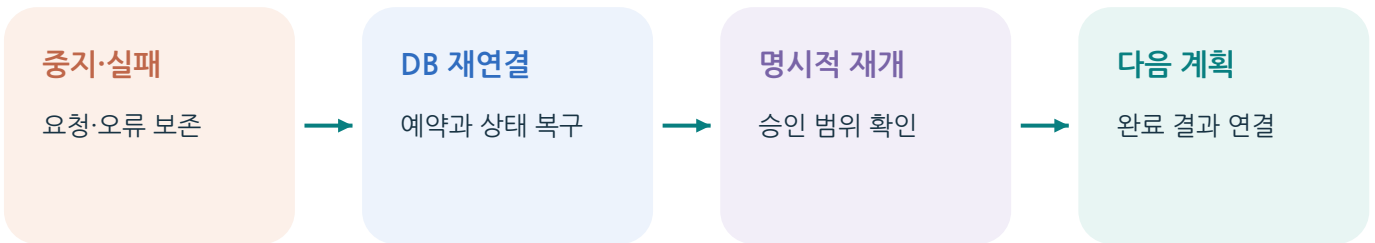
파일럿의 외부 자료는 저장된 시험용 스냅샷이었다. 실제 Slack·Jira·Figma를 모두 연결한 종단 간 검증은 포함하지 않는다. 세션별 읽음 기록은 확인했지만 실제 재전송 전체를 계측하지 않아 “중복 전송 0회”라고 결론 내릴 수 없다. 예산과 리뷰 횟수 추가도 있었으므로 완전 무인 실행으로 소개하지 않는다.

9월 27일 검사: 개발본 1,719 · Python 160 · 공개본 1,381. 새 엔진 검사와 별도 집계.

복구 계약의 검증과 남은 과제

멈춘 지점의 복구부터 다음 단계까지 확인하고, 운영 환경에서의 비교는 별도 수행.

10월 개편의 회귀 검사는 실제 Git과 공개 API를 사용했다. 자료 요청 도중 중지·실패를 만들고 DB를 다시 연 뒤, 승인된 작업의 재개·리뷰·로컬 완료·다음 계획을 연결했다. 모델 응답은 대역이며, 미수신과 접근 오류는 후속 목록에 남는지 확인했다.



검증 확장: 실제 외부 서비스 · 물리적 재전송 · 동일 조건 비용 비교 · 기록 손상 복구

복구 이후 실제 진행 확인

운영 서버에는 검토한 코드를 설치하고 기존 계획·체크포인트·배정·한도를 보존한 재시작을 확인했다. 실제 근거 영향 검토도 재개했다. 이 실행의 재개와 대상 앱의 통합 검증 완료는 구분한다.

다음 검증 범위와 설계 기준

이 작업에서 얻은 기준은 AI가 오래 실행됐다는 사실보다 무엇을 읽고, 무엇을 승인받아, 어떤 코드를 검증해 전달했는지 설명할 수 있어야 한다는 것이다. 기능을 추가할 때도 이 연결을 유지하면서 필요한 검증만 확장한다.